

연구논문/작품 중간보고서

2026 학년도 제 2 학기

제목	GPU 프로파일링·시뮬레이션 기반 LLM 추론 attention 커널 동적 선택 시스템	○ 논문() 작품(✓) ※해당란 체크
GitHub URL	https://github.com/sunjae0224/kernelscope (design-1-3 브랜치)	
평가등급	지도교수 수정보완 사항	팀원 명단
A,B,F 중 택 1 (지도교수가 부여)	○ ○ ○	이 선 재 (인) (학번:)

2026 년 9 월 27 일

지도교수 : o o o 서명

■ 요약

대규모 언어 모델(LLM) 서빙 시스템은 GPU 활용률을 높이기 위해 길이가 서로 다른 요청을 하나의 배치로 묶어 처리하는 연속 배치(continuous batching)를 사용한다. 디코드 단계의 attention 연산에서는 요청마다 KV 캐시의 길이가 다르며, FlashAttention 계열 커널은 KV 캐시를 여러 조각으로 나누어 병렬로 계산하는 분할 수(num_splits)를 라이브러리의 휴리스틱으로 결정한다. 본 작품은 이 휴리스틱이 배치 안의 요청 길이가 고르다는 가정 위에서 동작하기 때문에 긴 요청과 짧은 요청이 섞인 배치에서는 GPU의 스트리밍 멀티프로세서(SM)를 충분히 활용하지 못한다는 점에 주목하였다.

RTX 4090에서 교체 가능한 attention 커널 변형을 6,801개 조건에서 측정한 결과, 균일한 길이의 배치에서 휴리스틱의 선택 손실은 중앙값 0.72%, 최대 5.71%에 그쳤다. 반면 길이가 서로 다른 요청이 섞인 가변 길이 배치에서는 162개 조건 중 124개에서 10% 이상의 손실이 나타났고, 최악의 경우 최적 설정보다 12.53배 느린 커널이 선택되었다. 이 현상의 원인을 스레드 블록(CTA) 단위 작업량 분석, 사이클 수준 GPU 시뮬레이터 Accel-Sim, 실측 데이터로 보정한 성능 모델로 설명하고, 측정값 조회 테이블(lookup table)과 성능 모델에 기반한 분할 수 선택 정책을 구현하였다.

선택 정책을 실제 Qwen3-4B 모델의 생성 과정에 적용한 결과, 32K 토큰 요청 1개와 512 토큰 요청 31개가 섞인 배치에서 요청별 토큰 간 지연(TPOT)이 61.06 ms에서 33.80 ms로 1.81배 감소하였으며, 5회 반복 모두 생성 토큰이 기본 설정과 동일하였다. 균일 길이 배치에서는 개선이 없었고, 일부 조건에서는 생성 토큰이 달라지는 한계도 확인하였다. 또한 GPU가 없는 환경에서도 저장된 원본 측정 기록으로부터 문서의 주요 수치 34개와 일관성 검사 3개를 다시 계산하는 재현성 검증 기능을 구현하였고, 37개 항목이 모두 일치함을 확인하였다. 중간 시점 기준으로 측정, 원인 분석, 선택, 실제 모델 검증의 전 과정이 동작하며, 남은 기간에는 출력 불일치 조건의 원인 분석, 성능 모델의 선택 손실 개선, 시뮬레이터 보정을 진행한다.

주요어: 대규모 언어 모델 추론, 연속 배치, FlashAttention, KV 분할, GPU 점유율, GPU 시뮬레이터

■ 1. 서론

가) 제안배경 및 필요성

Transformer[1] 구조에 기반한 대규모 언어 모델이 검색, 대화, 코드 작성 등 다양한 서비스에 활용되면서, 모델을 학습시키는 비용뿐 아니라 이미 학습된 모델로 답을 생성하는 추론(inference) 비용이 서비스 운영의 핵심 비용이 되었다. LLM의 추론은 입력 문장 전체를 한 번에 처리하는 프리필(prefill) 단계와, 이후 토큰을 하나씩 생성하는 디코드(decode) 단계로 나뉜다. 디코드 단계는 생성할 토큰 수만큼 반복되므로 전체 생성 시간의 대부분을 차지하며, 사용자는 이를 토큰 간 지연(TPOT, time per output token)으로 체감한다.

GPU 한 장이 처리할 수 있는 연산량에 비해 요청 하나의 디코드 연산은 매우 작기 때문에, 서빙 시스템은 여러 요청을 하나의 배치로 묶어 처리한다. Orca[2]는 요청 단위가 아니라 디코드 반복(iteration) 단위로 배치를 재구성하는 스케줄링을 제안하였고, vLLM[3]은 요청마다 길이가 다른 KV 캐시를 고정 크기 페이지 단위로 관리하는 PagedAttention을 제안하였다. 이러한 연속 배치 방식에서는 서로 다른 시점에 도착하여 서로 다른 길이의 문맥을 가진 요청들이 같은 배치 안에서 함께 처리된다. 실제 서비스에서는 긴 문서를 요약하는 요청과 짧은 대화 요청이 섞이는 것이 일반적이므로, 한 배치 안의 KV 캐시 길이는 수십 배 이상 차이가 날 수 있다.

디코드 단계의 attention은 새로 생성할 토큰 하나의 질의(query) 벡터가 지금까지 저장된 모든 키(key)와 값(value)을 읽어 가중합을 구하는 연산이다. 한 번 읽은 데이터에 대해 수행하는 연산이 적기 때문에 메모리 대역폭에 의해 성능이 제한되며, 병렬로 처리할 수 있는 작업의 양은 질의 길이가 아니라 배치 크기와 헤드 수에 의해 결정된다. 배치가 작거나 헤드 수가 적으면 GPU의 스트리밍 멀티프로세서(SM)를 모두 채우지 못한다. Flash-Decoding[4]은 이 문제를 해결하기 위해 각 요청의 KV 캐시를 여러 조각으로 나누어 서로 다른 스레드 블록이 병렬로 처리한 뒤 결과를 합치는 KV 분할(split-KV) 방식을 제안하였고, 이 방식은 FlashAttention-2[5] 라이브러리의 디코드 경로에 포함되어 널리 사용되고 있다.

이때 KV 캐시를 몇 조각으로 나눌지(분할 수, num_splits)는 라이브러리가 배치 크기, 헤드 수, 최대 KV 길이만을 보고 휴리스틱으로 정한다. 이 휴리스틱은 배치 안의 모든 요청이 최대 길이만큼의 작업을 가진다고 가정한다. 따라서 실행되는 스레드 블록의 수가 SM 수의 일정 비율을 넘으면 GPU가 이미 충분히 채워졌다고 판단하여 분할을 하지 않는다. 그러나 길이가 섞인 배치에서는 소수의 긴 요청을 맡은 스레드 블록에 작업이 몰리고, 짧은 요청을 맡은 블록은 금방 끝나 나머지 SM이 쉬게 되는 부하 불균형(load imbalance)이 발생한다. 이러한 비효율은 평균 지연 시간과 같은 시스템 수준의 지표만으로는 원인을 알기 어렵고, 커널 수준의 측정과 GPU 실행 구조에 대한 분석이 함께 필요하다.

또한 한 종류의 GPU에서 얻은 측정만으로는 SM 수나 메모리 대역폭이 다른 하드웨어에서 같은 현상이 어떻게 나타날지 답할 수 없다. 이를 위해서는 GPGPU-Sim[6], Accel-Sim[7]과 같은 사이클 수준 GPU 시뮬레이터나 실측으로 보정한 성능 모델을 활용하여 하드웨어 조건을 바꾸어 보는 민감도 분석(sensitivity analysis)이 필요하다. 이상을 정리하면 본 작품의 필요성은 다음과 같다.

- 1) 실제 서빙 조건인 가변 길이 배치에서 라이브러리의 커널 설정 선택이 얼마나 손해를 보는지 정량적으로 측정할 필요가 있다.

- 2) 그 원인을 스레드 블록 구성과 SM 점유율, 메모리 대역폭 등 GPU 실행 구조의 관점에서 설명할 필요가 있다.
- 3) 커널 수준의 개선이 실제 사용자가 체감하는 토큰 간 지연으로 이어지는지, 전체 모델의 생성 과정에서 검증할 필요가 있다.
- 4) 설정 변경이 생성 결과를 바꾸지 않는지, 즉 생성 결과가 동일한지를 성능과 함께 검증할 필요가 있다.

나) 연구논문/작품의 목표

본 작품의 최종 목표는 **요청 길이 분포에 따라 attention 커널의 분할 수를 매 디코드 단계마다 동적으로 선택하는 시스템**을 구현하고, 그 효과를 실제 LLM의 생성 과정에서 검증하는 것이다. 새로운 attention 수식이나 새로운 커널을 만드는 것이 아니라, 수학적으로 같은 결과를 내는 기존 커널의 실행 설정만을 바꾼다는 점이 특징이다. 이 원칙에 따라 성능 개선은 생성 토큰이 기본 설정과 동일한 경우에만 개선으로 인정한다. 세부 목표는 표 1 과 같다.

표 1. 세부 목표와 중간 시점 진행 상황

구분	세부 목표	중간 시점 결과	상태
측정	교체 가능한 attention 커널 변형(분할 수 1~128, 연속/페이지 단위 KV 캐시)을 같은 입력으로 측정하는 도구	RTX 4090 에서 8 개 측정 집합, 6,801 개 조건, 오류 0 건	완료
원인 분석	스레드 블록 구성·점유율 분석, 사이클 수준 시뮬레이터, 실측 보정 성능 모델로 원인 설명 및 민감도 분석	휴리스틱의 가정과 부하 불균형 기전 확인, Accel-Sim 43 건 재실행, 성능 모델 검증	진행 중
선택 정책	기본 휴리스틱, 고정 분할 수, 조회 테이블, 성능 모델 정책 구현	4 종 정책 구현, 성능 모델 첫 선택 비용 59 배 단축	완료
실제 모델 검증	같은 가중치·입력·요청 순서에서 정책만 바꾸어 TPOT 과 생성 토큰 일치 측정	Qwen3-4B TPOT 1.81 배 개선, 2 개 모델 후속 실험 108 회	진행 중
재현·시각화	GPU 없이 원본 기록으로 결과를 다시 계산하고 대시보드로 탐색	재현성 검증 37 개 항목 일치, 4 개 화면 대시보드	진행 중

다) 연구논문/작품 전체 overview

그림 1 은 제안하는 시스템의 전체 구성이다. 입력은 요청 길이 분포, 즉 한 배치 안에 어떤 길이의 요청이 몇 개 있는지이다. ① 커널 실측 단계에서는 FlashAttention-2 기반의 교체 가능한 커널 변형을 RTX 4090 에서 측정하여 조건별 실행 시간과 스레드 블록 수를 기록한다. ② 원인 설명과 예측 단계에서는 스레드 블록 구성과 점유율 분석, Accel-Sim 을 이용한 하드웨어 민감도 분석, 실측으로 보정한 성능 모델로 어떤 조건에서 왜 느려지는지를 설명한다. ③ 커널 선택 정책은 이 결과를 바탕으로 매 디코드 단계의 분할 수를 고른다. ④ 실제 LLM 생성 단계에서는 Qwen3-4B 와 Llama 계열 8B 모델을 연속 배치와 페이지 단위 KV 캐시로 실행하면서 정책만 교체한다. ⑤ 결과 검증 단계에서는 TPOT 과

생성 토큰의 일치 여부를 반복 측정하고, ⑥ 재현·시각화 단계에서는 GPU 없이 원본 기록으로 수치를 다시 계산하고 대시보드로 결과를 탐색한다.

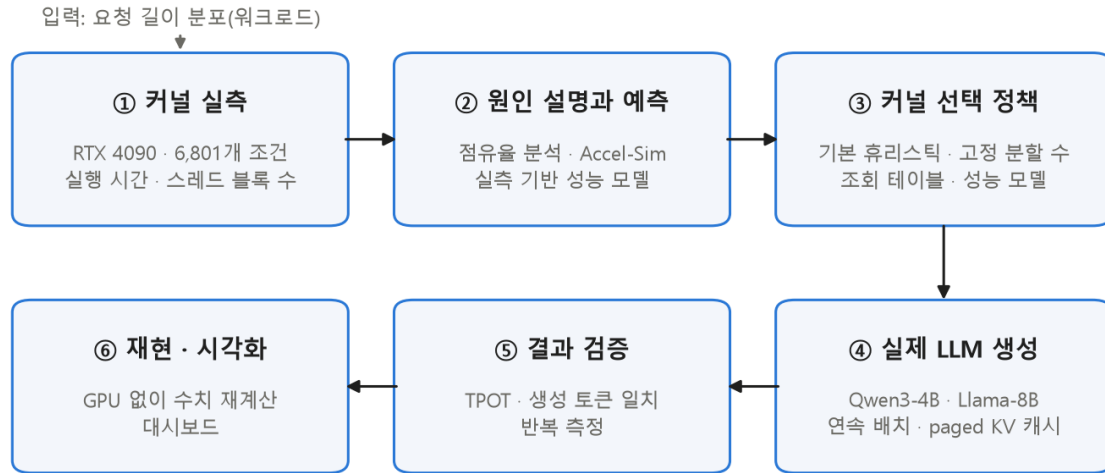


그림 1. 제안 시스템의 전체 구성

중간 시점까지 얻은 주요 결과는 다음과 같다. 첫째, 커널 수준에서 균일 길이 배치의 휴리스틱의 선택 손실은 중앙값 0.72%로 작았으나, 가변 길이 배치에서는 손실 중앙값이 172%(2.72 배)였고 최악의 경우 12.53 배 느린 설정이 선택되었다. 둘째, 실제 Qwen3-4B 생성에서 조희 테이블 정책은 가변 길이 배치의 TPOT 을 1.81 배, 요청 도착 시나리오(요청이 실행 도중 도착)의 TPOT 을 1.17 배 줄였고 생성 토큰은 동일하였다. 균일 길이 배치에서는 개선이 없었는데, 이는 커널 측정에서 휴리스틱이 이미 최적에 가까웠다는 결과와 일치한다. 셋째, 두 모델과 직접 작성한 자연어 입력으로 수행한 후속 실험 108 회에서 성능 모델 정책은 가변 길이 시나리오의 TPOT 을 Qwen3-4B 에서 1.28~1.29 배, Llama 계열 8B 모델에서 1.18~1.19 배 줄였다. 넷째, 성능 모델의 실행 시간 예측 오차 중앙값은 2.3~12.3%로 목표 안에 들었지만, 최악의 경우 선택한 커널이 최적보다 24~66% 느려 선택 손실 기준은 통과하지 못하였다. 다섯째, GPU 가 없는 노트북에서 문서의 주요 수치를 원본 기록으로부터 다시 계산하는 검증 기능을 구현하여 37 개 항목이 모두 일치함을 확인하였다.

본 보고서의 구성은 다음과 같다. 2 장에서는 LLM 서빙 시스템, attention 커널 최적화, GPU 성능 분석과 시뮬레이션에 관한 관련 연구를 정리한다. 3 장에서는 이론적 배경과 시스템의 각 구성 요소, 원안 대비 변경 사항을 설명한다. 4 장에서는 구현 현황과 실험 환경, 커널 수준과 실제 모델 수준의 결과, 성능 모델과 시뮬레이터의 검증 결과, GPU 없는 환경의 재현성 검증 결과를 분석하고 한계와 보완 계획을 제시한다. 5 장에서는 결론과 소감을 서술한다.

■ 2. 관련연구

2.1 LLM 추론과 서빙 시스템

Transformer[1]는 순환 구조 없이 attention 만으로 시퀀스를 처리하는 구조로, 현재 대부분의 대규모 언어 모델의 기반이 되었다. 자기회귀 방식으로 토큰을 생성할 때에는 이전 토큰들의 키와 값을 KV 캐시에 저장해 두고 재사용한다. 본 작품의 실험에 사용한 Qwen3[8]는 0.6B 부터 235B 규모까지의 dense 및 MoE 모델로 구성된 최신 공개 모델 계열로, 여러 질의 헤드가 하나의 키값 헤드를 공유하는 grouped-query attention(GQA) 구조를 사용한다.

Orca[2]는 요청 전체가 끝날 때까지 기다리지 않고 디코드 반복 단위로 배치를 다시 구성하는 반복 단위 스케줄링(iteration-level scheduling)을 제안하여, 길이가 서로 다른 생성 요청을 효율적으로 함께 처리할 수 있게 하였다. vLLM[3]은 운영체제의 가상 메모리처럼 KV 캐시를 고정 크기의 블록 단위로 관리하는 PagedAttention 을 제안하여 가변 길이 KV 캐시로 인한 메모리 낭비를 크게 줄였다. Sarathi-Serve[9]는 긴 프리필 요청을 여러 조각으로 나누어 진행 중인 디코드와 함께 처리하는 스케줄링으로 처리량과 지연 시간 사이의 균형을 개선하였다. 이들 연구가 확립한 연속 배치와 페이지 단위 KV 캐시 구조가 곧 본 작품이 다루는 가변 길이 배치가 발생하는 환경이다.

국내에서도 LLM 서빙의 성능 문제를 다룬 연구가 활발하다. 김수진 외[10]는 질의 간 의미적 유사도로 요청을 군집화하여 KV 캐시를 미리 구축하고 재사용함으로써 응답 시간과 처리량을 개선하였다. 최우형 외[11]는 LLM 서빙에서 GPU 메모리 용량의 한계를 극복하기 위해 GPU 메모리, 호스트 메모리, 저장장치를 계층적으로 활용하는 사례들을 정리하였다. 정선욱 외[12]는 다중 GPU 시스템에서 복잡한 딥러닝 모델을 고성능으로 추론하기 위한 스케줄링 전략을 비교하였고, 박시형 외[13]는 주요 상용 LLM 가속기를 메모리 구조, 성능, 소프트웨어 관점에서 비교하였다. 이 연구들이 무엇을 캐시에 둘지, 메모리를 어떻게 나눌지, 여러 장치에 작업을 어떻게 배치할지를 다룬다면, 본 작품은 이미 구성된 배치를 단일 GPU 에서 계산할 때 attention 커널 하나의 실행 설정을 어떻게 정할지라는 더 세밀한 단위의 문제를 다룬다.

2.2 attention 커널 최적화

FlashAttention[14]은 GPU 의 고대역폭 메모리(HBM)와 온칩 SRAM 사이의 데이터 이동을 최소화하는 타일링 기법으로, 근사 없이 정확한 attention 을 빠르고 메모리 효율적으로 계산하였다. FlashAttention-2[5]는 스레드 블록과 워프 사이의 작업 분할 방식을 다시 설계하여 GPU 점유율을 높이고 처리량을 개선하였다. Flash-Decoding[4]은 디코드 단계에서 질의 길이가 1 이기 때문에 배치가 작으면 SM 을 충분히 활용하지 못하는 문제를 지적하고, KV 캐시를 여러 조각으로 나누어 병렬로 계산한 뒤 log-sum-exp 방식으로 부분 결과를 합치는 방법을 제안하였다. 본 작품이 선택하는 대상인 분할 수가 바로 이 방법의 핵심 설정값이다.

FlashDecoding++[15]는 부분 softmax 결과를 합칠 때의 동기화 비용, 작은 행렬 곱의 낮은 활용률, 고정된 실행 방식의 비효율을 각각 unified max value 기법, flat GEMM 최적화, 입력 형태와 하드웨어에 따른 휴리스틱 선택으로 해결하였다. FlashInfer[16]는 다양한 KV 캐시 형식을 블록 희소 행렬 형식으로 통일하고, 요청별 작업량 차이를 고려하는 부하 분산 스케줄링과 JIT 컴파일 기반의 attention 템플릿을

제공하는 추론 엔진이다. 특히 FlashInfer는 길이가 다른 요청이 섞인 배치의 부하 불균형을 커널 내부의 스케줄링으로 해결한다는 점에서 본 작품과 목표가 가장 가깝다. 본 작품은 커널을 수정하지 않고 기존 FlashAttention-2 커널의 분할 수 인자만 외부에서 선택하므로, 이미 배포된 라이브러리 위에 적용할 수 있고 설정 변경 전후의 출력 동일성을 직접 검증할 수 있다는 차이가 있다. FlashInfer와의 정량 비교는 향후 과제로 남겨 둔다.

2.3 GPU 성능 분석과 시뮬레이션

루프라인 모델(roofline model)[17]은 연산 성능, 연산 강도(operational intensity), 메모리 대역폭을 하나의 그래프로 결합하여 프로그램이 연산 한계와 대역폭 한계 중 어디에 묶여 있는지를 판단하게 해준다. 본 작품에서는 각 커널의 이동 바이트와 연산량을 해석적으로 계산한 뒤 실측한 최대 대역폭 연산 성능과 비교하는 데 이 모델을 사용한다. GPGPU-Sim[6]은 NVIDIA의 PTX(parallel thread execution) 가상 명령어를 실행하는 상세한 GPU 마이크로아키텍처 시뮬레이터로, 인터커넥트, 캐시, 메모리 컨트롤러 등 설계 요소가 성능에 미치는 영향을 분석할 수 있게 하였다. Accel-Sim[7]은 실제 GPU에서 수집한 기계어(SASS) 명령어 추적을 입력으로 사용하는 trace 기반 시뮬레이션과, 마이크로벤치마크와 자동 파라미터 조정으로 구성된 검증 방법론(validation methodology)을 제공하여 최신 GPU를 모델링할 수 있게 한 프레임워크이다. 본 작품은 Accel-Sim에 RTX 4090 구성을 적용하여 SM 수, 메모리 대역폭, L2 캐시 크기를 바꾸는 민감도 분석을 수행하였다.

국내 연구로는 최신 GPU의 데이터 캐시 동작을 마이크로벤치마크로 측정하여 캐시 계층 구조에 따른 성능 특성을 분석한 정종현 외[18], 딥러닝 연산에 대해 GPU와 CPU의 실행 성능을 실측하여 비교한 최진서 외[19], 메모리 요청 상태를 관리하는 MSHR(miss status handling register)의 정보를 이용해 워프 스케줄링을 동적으로 조정함으로써 GPU의 명령어 처리율을 높인 김광복 외[20]가 있다. 이 연구들은 GPU 내부의 캐시와 스케줄링이 자원 활용률과 성능에 미치는 영향을 실측과 미세 구조 수준에서 분석하였다는 점에서, 본 작품이 분할 수 선택에 따른 SM 활용률 변화를 측정하고 설명하는 방법론적 근거가 된다.

2.4 원안과 관련된 연구 및 본 작품의 위치

본 작품의 원안은 Transformer와 선택적 상태 공간 모델인 Mamba[21] 사이를 입력에 따라 동적으로 전환하는 시스템이었다. Mamba는 시퀀스 길이에 선형으로 비례하는 계산량으로 Transformer에 견줄 만한 성능을 보였으나, 두 모델은 구조와 가중치가 달라 요청을 다른 모델로 보내면 지연 시간뿐 아니라 생성 결과 자체가 달라진다. 이 점이 3.3 절에서 설명하는 연구 대상 변경의 계기가 되었다. 표 2는 관련 연구와 본 작품의 대상 수준과 접근 방법을 비교한 것이다.

표 2. 관련 연구와 본 작품의 비교

연구	대상 수준	접근 방법	본 작품과의 관계
Orca[2], vLLM[3]	서빙 시스템	반복 단위 스케줄링, 페이지 단위 KV 캐시 관리	가변 길이 배치가 발생하는 실행 환경을 제공
Sarathi-Serve[9]	서빙 스케줄러	프리필 분할과 디코드 동시 처리	배치 구성 수준의 최적화로 본 작품과 상호 보완

연구	대상 수준	접근 방법	본 작품과의 관계
Flash-Decoding[4], FlashAttention-2[5]	attention 커널	KV 분할과 부분 결과 결합, 분할 수 휴리스틱	본 작품이 선택하는 분할 수의 정의와 기본 선택 방식
FlashDecoding+ +[15]	추론 엔진	입력 형태에 따른 커널 실행 방식의 휴리스틱 선택	입력에 따라 실행 설정을 고른다는 문제의식이 같음
FlashInfer[16]	attention 엔진	커널 내부의 부하 분산 스케줄링	같은 문제를 커널 수정으로 해결, 향후 비교 대상
Accel-Sim[7], 루프라인[17]	성능 분석	사이클 수준 시뮬레이션, 대역폭·연산 한계 분석	원인 설명과 민감도 분석 도구로 활용
본 작품	커널 실행 설정	실측 테이블·성능 모델 기반 분할 수 동적 선택과 실제 모델 검증	커널 수정 없이 기존 라이브러리 위에 적용, 출력 동일성 검증

■ 3. 제안 작품 소개

3.1 이론적 배경

3.1.1 디코드 단계의 attention 연산

디코드 단계에서 요청 b 의 헤드 h 는 새 토큰의 질의 벡터 q (길이 d)와 지금까지 저장된 L_b 개의 키 행렬 K , 값 행렬 V 를 사용하여 $o = \text{softmax}(qK^T/\sqrt{d})V$ 를 계산한다. 실험에 사용한 Qwen3-4B는 질의 헤드 32개가 키·값 헤드 8개를 4개씩 공유하는 grouped-query attention 구조이며 헤드 차원 d 는 128이다. 한 디코드 단계에서 읽어야 하는 KV 캐시의 양은 요청 길이의 합에 비례하는 반면, 읽은 데이터 1바이트당 연산은 몇 번에 불과하다. 따라서 이 연산의 연산 강도는 GPU의 루프라인 꺾임점보다 훨씬 낮고, 실행 시간은 주로 메모리 대역폭과 병렬로 동작하는 스레드 블록의 수에 의해 결정된다.

3.1.2 GPU 실행 구조와 점유율

CUDA 커널은 여러 개의 스레드 블록(CTA)으로 이루어진 그리드로 실행된다. 각 CTA는 하나의 SM에 배치되며, 한 SM에 동시에 올라갈 수 있는 CTA의 수는 레지스터와 공유 메모리 사용량에 의해 제한된다. 이 비율을 점유율(occupancy)이라고 한다. 실험에 사용한 RTX 4090은 128개의 SM을 가진다. 커널 전체의 실행 시간은 가장 늦게 끝나는 SM에 의해 결정되므로, CTA마다 작업량이 크게 다르면 일부 SM만 끝까지 일하고 나머지는 쉬는 꼬리 구간이 생긴다. 이것이 본 작품이 다루는 부하 불균형이다.

3.1.3 KV 분할(split-KV)

KV 분할 방식에서는 각 (요청, 키·값 헤드) 쌍의 KV 캐시를 S 개의 구간으로 나누고, 서로 다른 CTA가 각 구간의 부분 attention을 계산한다. 각 CTA는 부분 결과와 함께 구간 내 최댓값과 지수합을 저장하고 별도의 결합(reduction) 커널이 log-sum-exp 방식으로 이를 합쳐 최종 결과를 만든다. 분할 수를 S 로 하면 CTA 수는 $S \times B \times H_{kv}$ 가 되어 병렬성이 커지지만, 부분 결과를 쓰고 다시 읽는 추가 메모리 이동과 결합 커널의 비용이 생긴다. 따라서 분할 수가 너무 작으면 GPU를 다 채우지 못하고, 너무 크면 결합 비용이 커지는 절충 관계가 있다.

3.1.4 라이브러리 휴리스틱과 그 가정

FlashAttention 2.8.3은 분할 수를 다음과 같이 정한다. 배치 크기 B , 키·값 헤드 수 H_{kv} , 질의 행 블록 수 m 으로 계산한 기본 CTA 수 $B \cdot H_{kv} \cdot m$ 이 SM 수의 2배(SM당 CTA 2개 가정)의 80% 이상이면 분할하지 않는다($S = 1$). 그렇지 않으면 1부터 128까지의 분할 수 중에서 SM 활용 효율이 최고 효율의 85% 이상인 가장 작은 값을 고른다. 본 작품은 이 규칙과 그리드 구성 방식을 라이브러리 소스 코드(flash_api.cpp)에서 그대로 옮겨 구현하여 각 조건의 분할 수와 CTA 구성을 계산한다(부록 A.1).

이 규칙은 모든 요청이 최대 KV 길이만큼 일한다고 가정한다. 예를 들어 32K 토큰 요청 1개와 512 토큰 요청 31개로 이루어진 배치($B = 32$)에서는 기본 CTA 수가 $32 \times 8 = 256$ 으로 기준값 204.8을 넘으므로 분할을 하지 않는다. 그러나 실제 작업은 32K 요청을 맡은 CTA 8개에 몰려 있다. 이 8개는 각각 32,768개의 키를 처리하고 나머지 248개는 512개만 처리하므로, 128개 SM 중 8개만 끝까지 작업하고 나머지는 대부분의 시간 동안 쉬게 된다. 그림 2는 이 경우와 측정상 최적인 분할 수 16의 CTA

별 작업량을 비교한 것이다. 분할 수 16에서는 CTA 하나가 처리하는 최대 키 수가 2,048 개로 줄어 작업이 여러 SM에 고르게 퍼지며, 실측 커널 시간은 3,501 μ s에서 279 μ s로 줄었다.

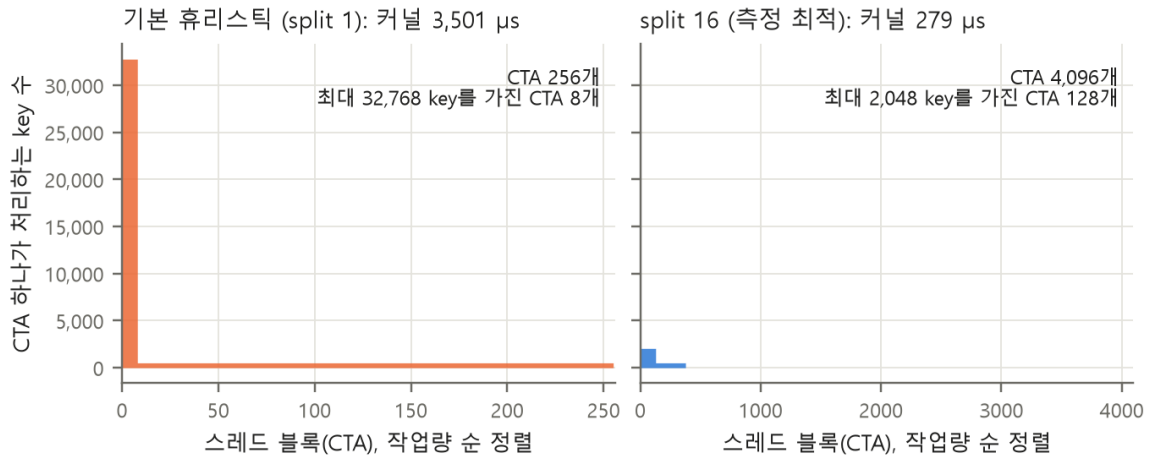


그림 2. 가변 길이 배치(32K×1 + 512×31)에서 분할 수에 따른 CTA 별 작업량 분포

3.1.5 성능 손실과 암달의 법칙

같은 계산을 하는 후보 커널 집합에서 어떤 선택의 **선택 손실(regret)**을 (선택한 커널의 실행 시간 / 같은 조건에서 가장 빠른 후보의 실행 시간) - 1로 정의한다. 후보 집합은 연속 KV 캐시와 페이지 단위 KV 캐시로 나누어 각각 따로 비교하며, 캐시를 비운 상태(cold)와 재사용 상태(warm)도 구분한다. 또한 attention이 한 디코드 단계에서 차지하는 비율이 f 이고 attention만 s 배 빨라질 때, 디코드 단계 전체의 속도 향상은 암달의 법칙에 따라 $1 / ((1 - f) + f / s)$ 를 넘을 수 없다. 이 관계는 4.3 절에서 커널 수준의 개선이 전체 생성 시간으로 얼마나 이어지는지 설명하는 데 사용한다.

3.2 시스템 구성 요소

시스템은 그림 1의 여섯 단계에 대응하는 구성 요소로 이루어지며, Python 패키지 약 7,200 줄과 대시보드·스크립트 약 1,700 줄로 구현하였다. 각 구성 요소는 파일 형식(Parquet, CSV, JSON)의 결과를 주고받으므로 GPU가 필요한 측정 단계와 GPU가 필요 없는 분석·재현 단계를 분리하여 실행할 수 있다.

(1) 커널 측정 도구

측정 대상 커널은 플러그인으로 등록한다. FlashAttention-2의 분할하지 않는 커널(분할 수 1), 라이브러리 휴리스틱을 사용하는 Flash-Decoding 경로(분할 수 0), 분할 수를 2~128로 고정한 변형 7종을 연속 KV 캐시와 페이지 단위 KV 캐시에 대해 각각 등록하여, 한 조건마다 9개의 교체 가능한 후보를 비교한다. 커널 실행 시간은 CUDA 이벤트와 torch.profiler(CUPTI activity API)로 측정하고, 그리드·블록 크기, 레지스터, 공유 메모리 사용량을 함께 기록하여 점유율을 계산한다. 공용 서버에서 GPU 모니터링 도구가 하드웨어 카운터를 점유하고 있어 Nsight Compute를 사용할 수 없었기 때문에, 하드웨어 카운터 없이 측정하도록 설계하였다. 캐시를 비운 상태의 측정에서는 매 측정 전에 L2 캐시 크기의 4배에 해당하는 데이터를 쓰는 커널을 실행하여 다른 레이어의 가중치가 L2를 지나간 실제 서빙 상황을 재현한다. 모든 측정은 참조 구현과 결과를 비교하여 정확성을 확인하고, 측정 시작 시점의 GPU 사용률과 다른 프로세스 목록을 함께 기록한다.

(2) 원인 분석: 실행 구성 재현, 루프라인 분석, Accel-Sim

라이브러리의 분할 수 휴리스틱과 그리드 구성 규칙을 소스 코드에서 옮겨, 각 조건에서 몇 개의 CTA가 만들어지고 각 CTA가 몇 개의 키를 처리하는지 계산한다(그림 2). 또한 커널이 옮기는 최소 바이트와 연산량을 해석적으로 계산하여 실측한 DRAM 대역폭(952.6 GB/s)과 텐서 코어 연산 성능(168.6 TFLOPS) 대비 달성률을 구한다. 하드웨어 조건을 바꾼 분석에는 Accel-Sim을 사용한다. 실제 GPU에서 NVBit로 커널의 명령어 추적을 수집한 뒤, RTX 4090 구성(SM 128개, L2 72 MiB, 384비트 메모리 버스)과 이를 변형한 구성(메모리 대역폭 2배·절반, L2 크기 2배·절반, SM 수 2배·절반)에서 재실행하여 어떤 자원이 성능을 제한하는지 판단한다.

(3) 실측 기반 성능 모델

측정하지 않은 조건이나 다른 하드웨어에서의 실행 시간을 빠르게 예측하기 위해 실측 기반 성능 모델을 구현하였다. 모델은 (2)에서 구한 CTA 별 키 수를 입력으로 받아, SM 수 $\times$ SM 당 동시 실행 가능한 CTA 수만큼의 실행 슬롯에 CTA를 배치하고, 동시에 실행되는 CTA들이 메모리 대역폭을 나누어 쓰는 프로세서 공유(processor sharing) 방식의 이산 사건 시뮬레이션으로 커널 완료 시간을 계산한다. 키 하나당 처리 시간, 시작 비용, 결합 커널 비용 등의 파라미터는 균일 길이 측정 데이터에 Nelder-Mead 방법으로 맞추었다. 초기 NumPy 구현은 후보 9개를 평가하는 첫 선택에 약 0.84초가 걸렸으나, 같은 계산을 C로 다시 구현하여 14.3 ms로 줄였다. 두 구현은 7개 기존 조건에서 예측값과 순위가 모두 같음을 확인하였다.

(4) 커널 선택 정책

매 디코드 단계마다 엔진은 현재 배치의 요청 길이 목록을 정책에 넘기고, 정책은 분할 수를 돌려준다. 구현한 정책은 표 3과 같다. 조회 테이블 정책과 성능 모델 정책은 같은 길이 구성이 반복될 때 다시 계산하지 않도록 페이지(256 토큰) 단위로 양자화한 길이 목록을 키로 하는 캐시를 사용하며, 정책 계산에 걸린 CPU 시간은 별도로 기록한다.

표 3. 구현한 분할 수 선택 정책

정책	선택 방법	특징
기본 휴리스틱	분할 수 0을 전달하여 FlashAttention 라이브러리의 규칙을 사용	비교 기준(baseline)
고정 분할 수	항상 같은 분할 수 N을 사용(예: 8)	단순하지만 조건에 적응하지 않음
조회 테이블	같은 헤드 구성의 실측 조건 중 (배치 크기, 최대 길이, 평균 길이)의 로그 공간에서 가장 가까운 조건을 찾아, 그 조건에서 가장 빨랐던 분할 수를 사용(최근접 이웃)	계산 비용이 매우 작음, 측정 범위 밖에서는 부정확할 수 있음
성능 모델	후보 9개의 실행 시간을 성능 모델로 예측하고 가장 짧은 후보를 선택	측정하지 않은 조건에도 적용 가능, 첫 계산 비용이 큼

(5) 실제 LLM 실행 엔진

정책의 효과를 전체 모델 수준에서 측정하기 위해 Llama와 Qwen3 계열 디코더를 직접 구현하였다. 모델 가중치는 Hugging Face 형식(safetensors)의 bf16(bfloat16, 16 비트 부동소수점) 값으로 읽고, KV 캐시는 256 토큰 단위의 페이지로 관리하며, attention은 flash_attn_with_kvcache 함수에 매 단계 정책이 고른 분할 수를 전달한다. 요청은 시나리오 파일에 정의된 논리적 도착 단계에 따라 배치에 합류하는 연속 배치 방식으로 처리되며, 모든 정책이 같은 가중치, 같은 입력, 같은 요청 순서와 도착 시점을 사용한다. 생성은 탐욕적 디코딩(greedy decoding)으로 고정된 길이만큼 수행한다. 매 단계의 attention 시간(CUDA 이벤트), 단계 전체 시간, 정책 계산 시간, 토큰별 생성 시각을 기록하여 TPOT을 계산하고, 정책마다 생성된 토큰을 기본 휴리스틱의 결과와 비교한다. 엔진의 수치적 정확성은 독립된 Hugging Face transformers 구현과 출력을 비교하여 따로 확인하였다.

(6) 재현성 검증과 대시보드

측정 결과는 원본 파일의 해시와 함께 이식 가능한 결과 아카이브(demo_data)로 저장된다. 이번 기간에는 이 기록만으로 문서의 주요 수치를 다시 계산하는 재현성 검증 명령(kernelscope verify)을 새로 구현하였다(4.5 절). 대시보드는 병목 진단, 조건별 최적 커널, 하드웨어 조건 변경, 실제 생성 결과의 네 화면으로 구성되며, 실측값과 모델 예측값을 구분하여 표시한다. 두 기능 모두 GPU 없이 동작한다.

3.3 원안 대비 변경 사항과 사유

원안은 'GPU 메모리 계층 프로파일링 기반 Transformer↔Mamba 동적 스위칭 시스템과 대시보드'였다. 프로파일링으로 워크로드 특성을 파악하고, 그에 따라 실행 대상을 동적으로 전환하며, 그 과정을 대시보드로 보여 준다는 골격은 그대로 유지하였다. 다만 다음 세 가지를 변경하였다.

- 1) **전환 대상의 구체화.** Transformer와 Mamba는 구조와 가중치가 다른 별개의 모델이므로, 요청을 다른 모델로 보내면 지연 시간이 아니라 답 자체가 달라진다. 서빙 최적화는 같은 요청에 같은 답을 내야 한다는 조건을 만족해야 하므로, 전환 대상을 같은 모델 안에서 수학적으로 동등한 attention 커널의 실행 설정(분할 수)으로 바꾸었다.
- 2) **하드웨어 시뮬레이터 활용 방식(지도교수 피드백 반영).** 초기 계획은 GPGPU-Sim으로 직접 작성한 축소 커널을 비교하는 것이었다. 그러나 실제 서빙에 쓰이는 FlashAttention-2 커널을 분석하기 위해, 실제 GPU의 명령어 추적을 사용하는 Accel-Sim으로 RTX 4090 구성을 모델링하는 방식으로 바꾸었다. 시뮬레이터의 절대 시간 보정에 한계가 확인되어, 실측으로 보정한 성능 모델을 함께 사용한다.
- 3) **측정 방법.** 공용 GPU 서버에서 Nsight Compute를 사용할 수 없어, CUDA 이벤트와 CUPTI activity 기반의 측정, 해석적 트래픽 모델, 명령어 추적으로 대체하였다.

■ 4. 구현 및 결과분석

4.1 구현 현황과 실험 환경

실험 환경은 표 4와 같다. GPU 측정은 연구실의 RTX 4090 서버에서 수행하였고, 분석과 재현성 검증, 보고서 그림 생성은 GPU가 없는 노트북에서도 수행하였다. 자동화된 테스트는 61개 파일로 구성된다. 이번 기간에 추가한 테스트를 포함하여 GPU가 없는 노트북에서 470개가 통과하였다(GPU 전용 12개 제외, transformers 등 선택 의존성이 없어 9개 생략, 2026년 9월 25일 실행). 이와 별도로 연구실 서버에서는 디코더와 KV 캐시의 GPU 테스트 7개가 통과하였다.

표 4. 실험 환경

항목	내용
GPU	NVIDIA GeForce RTX 4090 (SM 128개, L2 72 MiB), 실측 DRAM 대역폭 952.6 GB/s, 텐서 코어 168.6 TFLOPS
소프트웨어	PyTorch 2.8.0 (CUDA 12.8 빌드), FlashAttention 2.8.3.post1, CUDA 12.9, 드라이버 580.95.05
시뮬레이터	Accel-Sim v2.0.0, NVBit 1.8, RTX 4090 구성 파일(SM89)
모델	Qwen3-4B-Instruct-2507, DeepSeek-R1-Distill-Llama-8B (bf16)
커널 측정 조건	균일 길이: $B \in \{1, \dots, 64\}$, $L \in \{512, \dots, 32768\}$ / 가변 길이: $B \in \{16, \dots, 64\}$, 긴 요청 1~2개, 긴 길이 8K~16K~32K, 짧은 길이 512~1K~2K / 헤드 구성 2종
재현성 검증 환경	WSL2 노트북, 8 코어, 메모리 3.8 GiB, GPU 없음, CPU 용 PyTorch 2.8.0

4.2 커널 수준 측정 결과

8개의 측정 집합에서 총 6,801개 조건을 측정하였고 오류는 없었다. 표 5는 기본 휴리스틱의 선택 손실을 요약한 것이다. 균일 길이 배치에서는 손실 중앙값이 1% 내외이고 10%를 넘는 조건이 없어, 라이브러리의 규칙이 설계 가정대로 잘 동작하였다. 반면 가변 길이 배치에서는 손실이 크게 늘어났다. 연속 KV 캐시에서는 162개 조건 중 124개, 페이지 단위 KV 캐시에서는 127개에서 10%를 넘는 손실이 나타났다.

표 5. 기본 휴리스틱의 선택 손실 (cold, 별도 표기가 없으면 헤드 구성 S1: 질의 헤드 32 / 키값 헤드 8)

측정 집합	조건 수	손실 중앙값	손실 최댓값	10% 초과 조건
균일 길이, 연속 KV	49	0.7%	5.7%	0
균일 길이, 페이지 KV	49	1.2%	10.0%	0
가변 길이, 연속 KV	162	172.4%	1152.7%	124
가변 길이, 페이지 KV	162	43.6%	298.1%	127
가변 길이, 페이지 KV (헤드 구성 S2)	135	47.2%	432.3%	102

그림 3은 연속 KV 캐시에서 휴리스틱의 커널 시간을 최적 커널 시간으로 나눈 비율의 누적 분포이다. 균일 길이 배치는 모든 조건이 1.06 배 이내에 모여 있는 반면, 가변 길이 배치는 중앙값이 2.72 배이고 최대 12.53 배까지 넓게 퍼져 있다. 즉 같은 라이브러리 규칙이 요청 길이의 분포에 따라 거의 최적이 되기도 하고 매우 나쁜 선택이 되기도 한다.

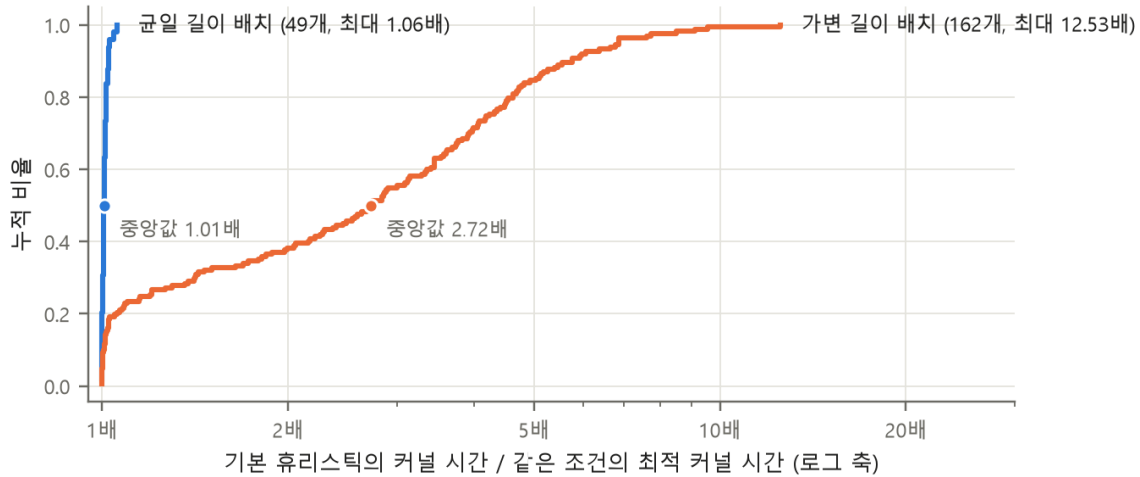


그림 3. 균일 길이와 가변 길이 배치에서 휴리스틱의 최적 대비 커널 시간 비의 누적 분포

그림 4는 손실이 가장 컸던 조건($32K \times 1 + 512 \times 31$)에서 분할 수에 따른 커널 시간이다. 분할 수가 커질수록 시간이 줄어들다가 8~16에서 최소가 되고, 그 이상에서는 결합 비용 때문에 다시 늘어난다. 두 경우 모두 휴리스틱은 분할 수 1을 선택하였다. 다만 페이지 단위 KV 캐시 경로는 분할 수가 1 이어도 항상 분할용 커널을 사용하므로 CTA 당 처리 속도가 달라, 손실이 연속 KV 캐시(12.53 배)보다 작은 3.84 배로 나타났다. 실제 LLM 엔진은 페이지 단위 KV 캐시를 사용하므로, 전체 모델에서 기대할 수 있는 개선의 크기는 뒤쪽 수치에 더 가깝다.

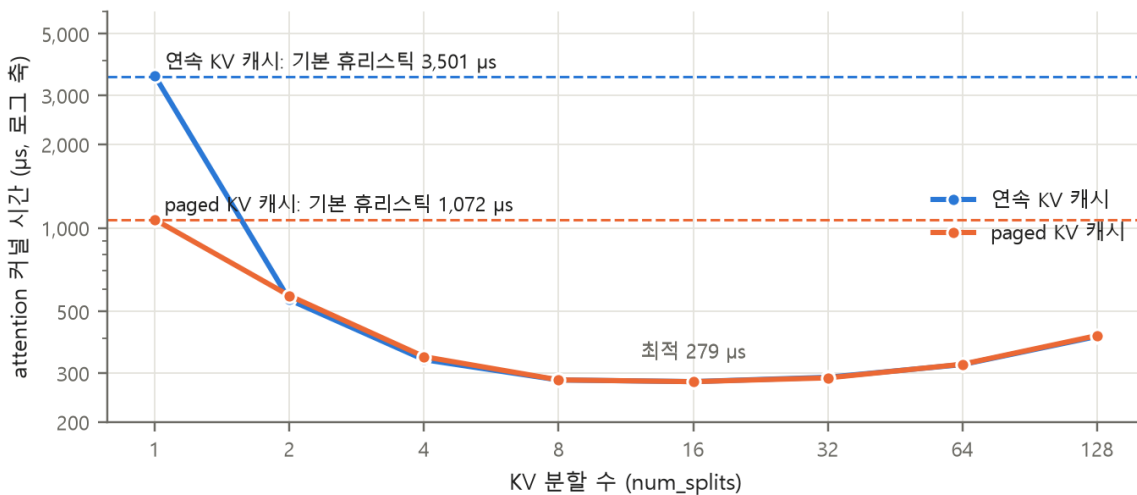


그림 4. 최악 조건($32K \times 1 + 512 \times 31$)에서 분할 수에 따른 attention 커널 시간

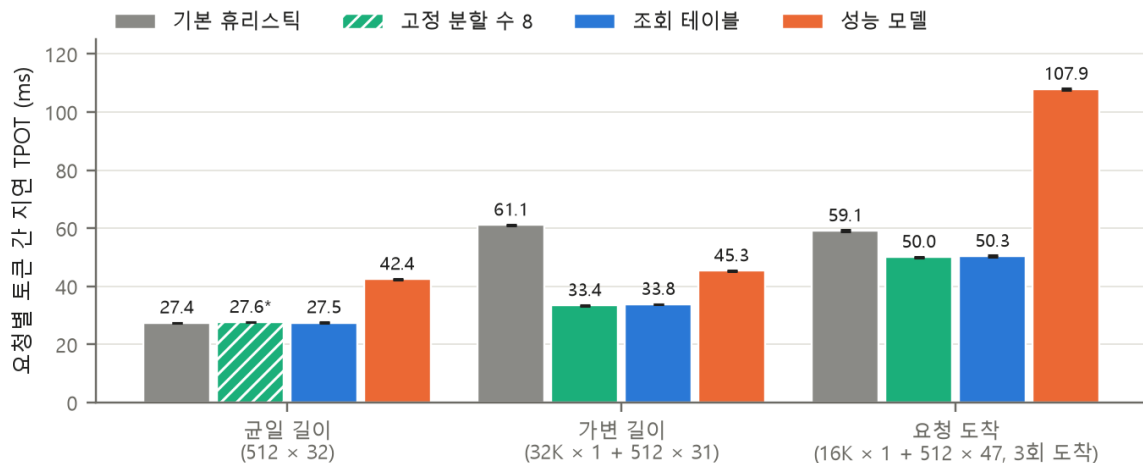
4.3 실제 LLM 생성 결과

Qwen3-4B 모델로 세 가지 시나리오를 실행하였다. 균일 길이는 512 토큰 요청 32 개, 가변 길이는 32K 토큰 요청 1 개와 512 토큰 요청 31 개이며, 두 경우 모두 요청마다 64 개 토큰을 생성한다. 요청

도착 시나리오에는 16K 요청 1 개와 512 토큰 요청 15 개로 시작하여 8 번째와 24 번째 단계에 512 토큰 요청이 16 개씩 추가로 도착한다. 정책마다 5 회 반복하였고, 반복마다 정책의 실행 순서를 바꾸어 순서에 따른 편향을 줄였다. 결과는 표 6 과 그림 5 와 같다.

표 6. Qwen3-4B 실제 생성 결과 (5 회 반복 평균, 개선 배율은 기본 휴리스틱 대비 TPOT 기준)

시나리오	정책	TPOT (ms)	attention (ms/ 단계)	디코드 (ms/ 단계)	개선 배율	토큰 일치
균일 길이	기본 휴리스틱	27.42	3.59	18.13	1.00	기준
	고정 분할 수 8	27.62	3.60	18.29	0.99	불일치 (94.43%)
	조희 테이블	27.50	3.58	18.15	1.00	일치
	성능 모델	42.43	3.59	33.06	0.65	일치
가변 길이	기본 휴리스틱	61.06	36.87	51.36	1.00	기준
	고정 분할 수 8	33.39	9.14	23.75	1.83	일치
	조희 테이블	33.80	9.24	24.08	1.81	일치
	성능 모델	45.33	9.13	35.63	1.35	일치
요청 도착	기본 휴리스틱	59.09	10.61	24.64	1.00	기준
	고정 분할 수 8	50.05	4.98	19.09	1.18	일치
	조희 테이블	50.34	5.04	19.23	1.17	일치
	성능 모델	107.87	5.00	56.22	0.55	일치



* 생성 토큰이 기본 휴리스틱과 달라 생성 결과가 동일한 개선으로 인정하지 않음

그림 5. 시나리오와 정책별 요청 토큰 간 지연(TPOT), 오차 막대는 반복 간 표준편차

가변 길이 시나리오에서 조희 테이블 정책은 단계당 attention 시간을 36.87 ms 에서 9.24 ms 로 약 4.0 배 줄였고, 그 결과 TPOT 이 61.06 ms 에서 33.80 ms 로 1.81 배 줄었다. 단계당 attention 이 차지하는 비율은 72%에서 38%로 낮아졌다. 이 개선이 attention 의 변화만으로 설명되는지 확인하기 위해, 기본 휴리스틱의 단계 시간(51.28 ms)에서 attention 시간 차이만큼을 바꾸어 예측한 값(23.64 ms)과 조희

테이블 정책의 실제 단계 시간(24.00 ms)을 비교하였다. 차이는 1.5%로, 나머지 연산(행렬 곱, 정규화 등)은 정책의 영향을 받지 않았고 개선은 attention 커널 선택에서 비롯되었음을 확인하였다. 요청 도착 시나리오에서는 긴 요청의 비중이 작아 개선 폭이 1.17 배로 작았다.

균일 길이 시나리오에서는 조회 테이블 정책의 개선이 없었다(0.997 배). 이는 4.2 절에서 균일 길이의 휴리스틱의 선택 손실이 1% 내외였다는 결과와 일치하며, 제안 방법이 모든 입력에서 이득을 주는 것이 아니라 요청 길이가 섞인 경우에만 의미가 있음을 보여 준다. 고정 분할 수 8 은 균일 길이에서 생성 토큰의 94.43%만 일치하였다. 분할 수가 바뀌면 부분 결과를 합치는 순서가 달라지고, bf16 연산의 반올림 오차 때문에 다음 토큰의 확률이 거의 같은 경우 선택되는 토큰이 바뀔 수 있기 때문이다. 이러한 결과는 생성 결과가 동일한 개선으로 인정하지 않았다. 성능 모델 정책은 커널 선택 자체는 좋았지만 (attention 9.13 ms), 새로운 길이 구성이 나올 때마다 수행하는 첫 계산이 NumPy 구현에서 수백 ms 가 걸려 균일 길이와 요청 도착 시나리오에서는 오히려 느려졌다. 이 문제는 성능 모델을 C 로 다시 구현하여 첫 선택 비용을 842.6 ms 에서 14.3 ms 로 줄이는 방식으로 개선하였다.

후속 실험에서는 조회 테이블과 모델 파라미터를 고정한 채, 두 모델(Qwen3-4B, Llama 계열 8B) × 직접 작성한 문단으로 구성된 자연어 입력 조건 3 개 × 난수 시드 2 개 × 정책 3 개 × 반복 3 회로 총 108 회를 측정하였다. 가변 길이 시나리오(12K 토큰 1 개 + 384 토큰 27 개)에서 성능 모델 정책은 TPOT 을 Qwen3-4B 에서 1.282~1.290 배, Llama 계열 8B 모델에서 1.184~1.188 배 줄였고 생성 토큰은 동일하였다. 균일 길이 시나리오에서는 개선이 없었다(0.98~1.00 배). 반면 요청 도착 시나리오의 모든 비교와 가변 길이의 조회 테이블 비교 2 건은 생성 토큰이 일치하지 않아, 기본 휴리스틱이 아닌 정책의 비교 24 건 중 14 건만 출력 검증을 통과하였다. 요청 도착 시나리오에서 입력 토큰을 고정한 채 다음 토큰 예측만 비교하는 진단에서는 정책별로 868 회 비교 중 866 회의 최대 확률 토큰이 일치하였다. 이는 불일치가 확률이 거의 같은 소수의 위치에서 시작되어 이후 생성이 달라진 것임을 시사하지만, 자유 생성에서의 출력 검증 실패를 대신하지는 않는다.

보고서 작성 시점에 기록된 108 회 실행을 다시 분석한 결과, 토큰이 달라진 10 개 실행은 모두 3 회 반복에서 같은 위치에서 불일치가 시작되었고, 불일치 시작 지점은 총 25 곳(그중 8 곳은 요청의 마지막 토큰)이었다. 교사 강요(teacher forcing) 방식의 진단에서 최대 확률 토큰이 뒤바뀐 4 개 위치의 상위 두 로짓 차이는 0.125 또는 0.25 로, 16~32 크기의 로짓을 bf16 이 표현하는 최소 간격(0.125)의 1~2 배였으며, 자유 생성에서 불일치가 시작된 4 개 지점은 모두 이 위치와 일치하였다. 즉 불일치는 특정 분할 수의 오류가 아니라 두 후보의 확률이 표현 정밀도 안에서 같은 위치에서 생기는 반올림 차이이다. 따라서 출력 검증 지표를 토큰 일치율에서 불일치 시작 지점의 수와, 그 지점의 상위 두 로짓 차이가 표현 정밀도 이내인지의 분류로 바꾼다.

4.4 성능 모델과 시뮬레이터 검증

성능 모델은 학습에 사용하지 않은 조건으로 네 가지 검증 집합을 구성하여 평가하였다. V1 은 균일 길이 측정 격자 사이의 보간 조건, V5 는 학습에 쓰지 않은 헤드 구성, V6 는 가변 길이 시나리오이다. 표 7 에서 보듯 실행 시간 예측 오차의 중앙값은 2.3~12.3%로 모든 집합에서 목표 기준을 통과하였다. 그림 6 은 가변 길이 검증 집합의 예측값과 실측값을 비교한 것으로, 대부분의 점이 $\pm 22.5\%$ 범위 안에 있으나 매우 긴 실행 시간의 일부 조건에서 모델이 시간을 절반 가까이 과소 예측하였다.

표 7. 성능 모델 검증 결과

집합	캐시	조건 수	예측 오차 중앙값 / 90% (기준)	시간 예측	선택 손실 중앙값 / 최댓값 (모델 휴리스틱)	선택 손실 (기준 5% / 15%)
V1	cold	594	9.0% / 22.5% (15% / 30%)	통과	0.0% / 24.3% 0.8% / 10.0%	미달
V1	warm	594	2.3% / 13.3% (15% / 30%)	통과	0.0% / 41.7% 1.0% / 41.7%	미달
V5	cold	882	12.3% / 32.1% (22% / 45%)	통과	0.0% / 66.3% 1.2% / 15.1%	미달
V6	cold	1,971	11.1% / 20.2% (22% / 45%)	통과	0.0% / 24.5% 64.2% / 1152.7%	미달

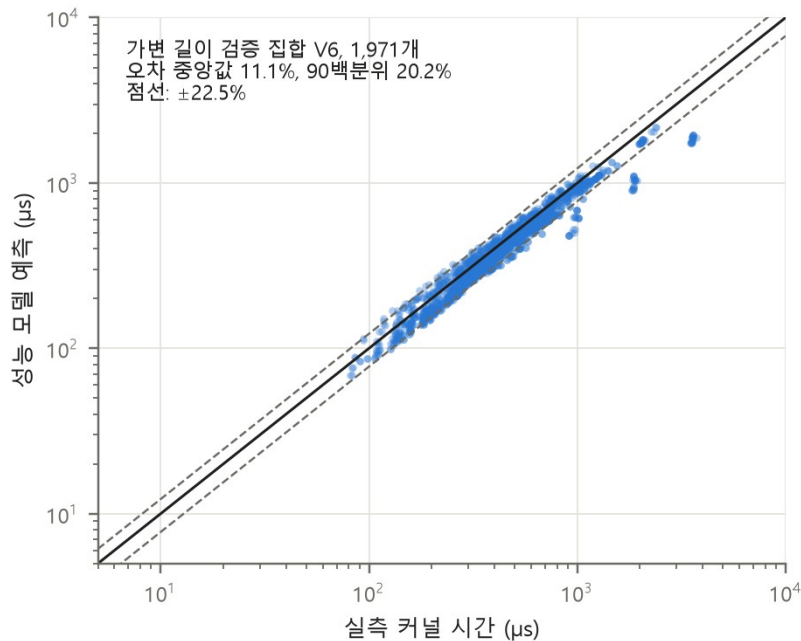


그림 6. 가변 길이 검증 집합(V6)에서 성능 모델의 예측값과 실측값 비교

선택 손실의 관점에서 보면, 모델이 고른 커널의 손실 중앙값은 모든 집합에서 0.0%로 대부분 최적을 골랐지만, 최악의 경우에는 24~66% 느린 커널을 골라 최대 손실 기준(15%)을 넘었다. 가변 길이 집합에서는 같은 조건의 기본 휴리스틱의 선택 손실(중앙값 64.2%, 최대 1152.7%)보다 훨씬 좋지만, 평균 예측 오차가 작다는 것만으로 항상 최적을 고른다고 말할 수는 없다. 따라서 현재는 측정 범위 안에서는 조회 테이블을 기본 정책으로 사용하고, 성능 모델은 측정하지 않은 조건과 하드웨어 민감도 분석에 사용한다.

보고서 작성 시점에 두 근거를 합치는 혼합 정책을 구현하였다. 성능 모델의 예측 순위를 따르되, 가장 가까운 실측 조건에서 가장 빨랐던 변형을 모델의 최선 예측값의 1.2 배 이내로 예측하면 그 변형을 고른다. 기록된 측정값에서 각 조건을 제외한 나머지로 테이블을 만드는 leave-one-out 방식으로 평가한 결과, 서버가 가정하는 cold 상태의 세 검증 집합에서 최악 손실이 각각 1.97%, 7.54%, 10.90%로

선택 손실 기준(15%)을 통과하였다. 모델 단독(24.3 / 66.3 / 24.5%)과 테이블 단독(50.4 / 66.1 / 100.7%)은 모두 기준을 넘었다. 이 결과는 GPU 없이 기록된 측정값으로 얻은 것이며, 실제 생성 과정에서의 검증은 아직 수행하지 않았다.

Accel-Sim 의 RTX 4090 모델은 표 8 과 같이 실측과 비교하였다. 작업량이 작고 CTA 가 8 개뿐인 FlashAttention-2 의 B = 1 조건에서는 시뮬레이션 시간이 실측의 1.04~1.06 배로 잘 맞았다. 그러나 Flash-Decoding 과 배치가 큰 조건에서는 1.5~3.1 배로 차이가 컸다. 원인을 조사한 결과, 사용한 L2 파티션 해시 설정에서 48 개 L2 슬라이스 중 16 개에 읽기 요청이 다른 슬라이스보다 약 2.0 배 물리는 편향을 확인하였다. 그러므로 현재 시뮬레이터의 결과는 절대 시간이 아니라 자원별 민감도의 경향을 보는 데에만 사용한다. 예를 들어 B = 16 조건에서 메모리 대역폭을 절반으로 줄이면 두 커널의 시간이 각각 97%, 102% 늘어나 대역폭에 의해 제한되는 상태임을 보였고, B = 1 의 FlashAttention-2 는 어떤 자원을 바꾸어도 시간이 거의 변하지 않아 CTA 수가 부족한 상태임을 보였다.

표 8. Accel-Sim RTX 4090 모델의 시뮬레이션/실측 시간 비

커널	조건	실측 (μ s)	시뮬레이션 사이클	시뮬레이션/실측
FlashAttention-2	B1, L 1K	63.58	170,012	1.061
FlashAttention-2	B1, L 8K	494.34	1,300,168	1.044
FlashAttention-2	B16, L 1K	64.30	276,533	1.707
Flash-Decoding	B1, L 1K	10.02	39,077	1.548
Flash-Decoding	B1, L 8K	27.46	185,450	2.680
Flash-Decoding	B16, L 1K	38.22	294,358	3.056
직접 작성한 FP32 커널	B1, L 1K	465.92	837,308	0.713

보고서 작성 시점에는 GPU 가 없는 개발용 노트북에서 GPGPU-Sim 4.2(구성 SM7_QV100, SM 80 개)를 빌드하여, 실측에서 추정된 부하 불균형 기전을 사이클 수준에서 확인하였다. 실제 FlashAttention-2 커널은 시뮬레이터에서 실행할 수 없으므로, 같은 구조(KV 캐시 용량 기준 분할, 부분 결과 결합)를 가진 fp32 축소 커널을 직접 작성하고 CPU 배정밀도 참조와 비교하여 정확성을 확인하였다(최대 오차 $2e-8$). 그림 7 은 키 총량이 거의 같은 두 배치, 즉 가변 길이 배치(2048 토큰 1 개 + 128 토큰 15 개)와 균일 길이 배치(256 토큰 16 개)에 대해 분할 수를 1 에서 16 까지 바꾼 시뮬레이션 사이클이다. 분할하지 않을 때 가변 길이 배치는 균일 배치보다 8.5 배 오래 걸리고 명령어 처리율(IPC)은 105 대 921 로 약 1/9 인데, 긴 요청을 맡은 스레드 블록 8 개가 2,048 개 키를 순차 처리하는 동안 나머지 SM 이 쉬기 때문이다. 분할 수를 늘리면 가변 길이 배치는 명령어 수가 43% 늘어남에도 16 에서 7.23 배 빨라지고, 균일 배치는 8 에서 2.25 배로 포화한 뒤 결합 비용 때문에 다시 느려진다. 이는 RTX 4090 실측(그림 4)과 같은 방향이다. 다만 이 축소 커널은 스레드 블록당 워프가 4 개뿐이어서 균일 배치도 분할로 이득을 얻는 등, 절대 사이클이나 최적 분할 수를 실측과 맞춘 것이 아니라 경향을 재현한 것이다.

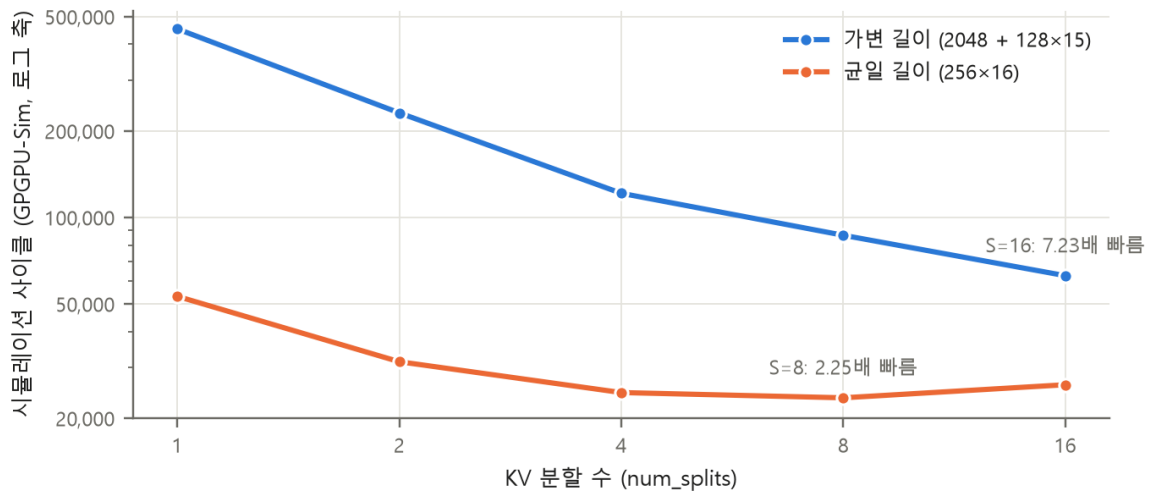


그림 7. GPGPU-Sim(SM7_QV100)에서 축소 split-KV 커널의 분할 수에 따른 시뮬레이션 사이클

4.5 GPU 없는 환경에서의 재현성 검증

현재 개발 환경인 노트북에는 GPU가 없고, 심사나 재현 시에도 같은 GPU를 준비하기 어렵다. 이번 기간에는 GPU 없이도 결과의 신뢰성을 확인할 수 있도록 재현성 검증 기능을 구현하였다. 이 기능은 저장된 원본 기록(커널 측정 7개 집합의 조건별 결과, 실제 생성 실험의 요약과 토큰 비교 결과, 후속 실험 요약, 선택 비용 측정 기록)만을 읽어, 프로젝트의 분석 코드로 각 수치를 다시 계산하고 문서에 적힌 값과 허용 오차 안에서 비교한다. 또한 해당 문서가 여전히 그 값을 적고 있는지도 확인하여, 데이터와 문서가 서로 어긋나는 것을 막는다. 성능 모델의 검증표도 원본 기록으로부터 다시 계산한다. 검사 그룹별 대상과 결과는 표 9와 같다. 이 과정에서 CPU용 PyTorch 환경에서 테스트 수집이 실패하던 문제(선택 의존성인 Triton의 즉시 로드)와, 이식 가능한 결과 아카이브를 분석 명령이 읽지 못하던 문제를 함께 수정하였다.

표 9. 재현성 검증 결과 (GPU 없는 노트북, 실행 시간 약 12 초)

검사 그룹	대상	항목 수	결과
커널 측정	균일 길이 손실 중앙값, 최댓값, 가변 길이 최악 조건의 휴리스틱/최적 비(연속-페이지 KV), 발표 예시 조건	5	모두 일치
실제 생성	세 시나리오의 TPOT, attention-디코드 시간, 개선 배율, 토큰 일치 여부와 일치율	12	모두 일치
후속 실험	측정 횟수, 두 모델의 개선 배율 범위, 출력 검증 통과 수	6	모두 일치
선택 비용	성능 모델 첫 선택 시간(NumPy, C 구현)	2	모두 일치
성능 모델	검증 집합별 예측 오차 중앙값, 가변 길이의 선택 손실 최댓값(모델, 휴리스틱), 혼합 정책의 손실 최댓값 3건	9	모두 일치
일관성	attention 시간 변화만으로 예측한 단계 시간과 실측의 차이 ($\pm 3\%$ 이내): 가변 길이 -1.5% , 요청 도착 -0.8% , 균일 길이 -0.05%	3	모두 통과

37 개 항목이 모두 일치하였다. 이로써 보고서와 발표에 사용하는 수치가 특정 장비에서만 확인 가능한 값이 아니라, 저장소에 포함된 원본 기록과 공개된 코드만으로 누구나 다시 계산할 수 있는 값임을 보였다. 한편 CPU 에서 작은 무작위 모델로 실행하는 엔진 시연은 스케줄링과 토큰 비교 로직만 확인할 수 있다. CPU 의 attention 구현은 분할 수를 사용하지 않으므로 커널 선택에 따른 성능 차이는 재현되지 않는다는 한계를 문서에 명시하였다.

4.6 한계와 보완 계획

현재 결과의 한계는 다음과 같다. 첫째, 모든 측정은 RTX 4090 한 종류에서 수행되었다. 둘째, 입력은 길이를 통제한 합성 토큰과 직접 작성한 문단이며, 실제 서비스의 요청 분포를 재현한 것은 아니다. 셋째, 요청 도착 시나리오 등에서 생성 토큰이 일치하지 않는 경우가 있다. 넷째, 성능 모델은 최악의 경우 선택 손실 기준을 통과하지 못하였다. 다섯째, Accel-Sim 모델의 절대 시간이 보정되지 않았다. 여섯째, 커널 내부에서 부하를 분산하는 FlashInfer 등 기존 방법과의 비교가 없다. 또한 본 엔진은 단일 프로세스 실험용 엔진이며, HTTP 서버, CUDA Graph, 다중 GPU 환경은 범위 밖이다. 남은 기간의 보완 계획은 표 10 과 같다.

표 10. 남은 기간의 보완 계획

보완 항목	내용	시기
출력 불일치 원인 분석	완료(9 월 26 일). 기록된 108 회 실행을 다시 분석한 결과 불일치는 상위 두 로짓의 차이가 bf16 최소 표현 간격의 1~2 배 이내인 위치에서 시작되는 결정적 차이였다. 검증 지표를 토큰 일치율에서 불일치 시작 지점의 수와 상위 두 로짓 차이의 분류로 바꾸고, 새 실험에서 이 분류를 자동으로 얻는 진단 실행이 남았다	GPU 확보 시
성능 모델의 선택 손실 개선	혼합 정책 구현 완료(9 월 26 일). 예측 차이가 20% 이내이면 가장 가까운 실측 조건의 답을 따르는 정책이 기록된 측정값의 leave-one-out 평가에서 최악 손실 1.97 / 7.54 / 10.90%로 기준(15%)을 통과. 실제 생성에서의 검증이 남았다	GPU 확보 시
시뮬레이터 보정과 활용	GPGPU-Sim 에서의 기전 재현은 완료(9 월 26 일, 4.4 절). Accel-Sim 의 L2 파티션 해시 설정을 바꾸어 재검증하는 작업이 남았다	GPU 확보 시
기존 방법과의 비교	FlashInfer 의 부하 분산 방식과 같은 조건에서 커널 시간과 TPOT 비교(연구실 GPU 사용 가능 시)	11 월
결과 정리와 발표 준비	대시보드에 재현성 검증 화면 추가, 최종 보고서 작성, 시연 준비	12 월

■ 5. 결론 및 소감

5.1 결론

본 작품은 LLM 서버에서 길이가 서로 다른 요청이 한 배치에 섞일 때, attention 커널의 분할 수를 정하는 라이브러리의 휴리스틱이 요청 길이가 고르다는 가정 때문에 GPU를 충분히 활용하지 못한다는 문제를 실측으로 확인하였다. RTX 4090의 6,801개 조건 측정에서 균일 길이 배치의 손실은 중앙값 0.72%였지만 가변 길이 배치에서는 최대 12.53배 느린 선택이 나타났고, 그 원인이 소수의 CTA에 작업이 몰리는 부하 불균형임을 CTA별 작업량 분석으로 설명하였다. 조회 테이블 기반 선택 정책을 실제 Qwen3-4B의 생성 과정에 적용하여, 가변 길이 배치의 TPOT을 생성 토큰 변화 없이 1.81배 줄였고, 그 개선이 attention 시간 변화로 1.5% 이내에서 설명됨을 확인하였다. 또한 개선이 없는 조건, 출력이 달라지는 조건, 성능 모델의 선택 실패 사례를 함께 보고하고, GPU 없이 모든 주요 수치를 다시 계산할 수 있는 검증 기능을 갖추었다. 남은 기간에는 출력 동일성 조건의 분석, 선택 손실 개선, 시뮬레이터 보정을 통해 결과의 신뢰도를 높일 계획이다.

5.2 소감

가장 크게 배운 점은 문제를 정확히 정의하는 일의 중요성이다. 처음에는 Transformer와 Mamba를 상황에 따라 바꾸는 시스템을 계획했지만, 두 모델은 서로 다른 답을 낸다는 사실을 깨닫고 '같은 답을 내면서 더 빠르게'라는 서버 최적화의 기본 조건으로 돌아가 문제를 다시 정의하였다. 그 결과 전환 대상을 같은 모델 안의 커널 설정으로 좁힐 수 있었고, 이것이 이후 모든 실험의 기준이 되었다.

측정은 생각보다 훨씬 어려웠다. 공용 GPU 서버에서 다른 작업이 GPU를 모두 사용하고 있던 날의 측정에서, 프로파일러가 기록한 커널 시간이 전체 지연 시간보다 세 배나 긴 물리적으로 불가능한 값이 나온 것을 뒤늦게 발견하였다. 그날의 측정을 모두 버리고 유휴 GPU에서 다시 측정하였으며, 이후 모든 측정에 GPU 사용률과 다른 프로세스 기록을 남기도록 바꾸었다. Nsight Compute를 사용할 수 없는 환경, 서버의 라이브러리 버전 때문에 FlashAttention을 직접 빌드해야 했던 일, 시뮬레이터의 결과가 실측과 맞지 않아 원인을 추적한 과정도 도구를 그대로 믿지 않고 검증하는 습관을 길러 주었다.

또 하나의 경험은 좋지 않은 결과를 그대로 보고하는 일이었다. 균일 길이에서는 개선이 없고, 일부 조건에서는 생성 토큰이 달라지며, 공들여 만든 성능 모델이 오히려 느려지는 결과를 처음에는 숨기고 싶었다. 그러나 이 결과들이 제안 방법이 언제 유효하고 언제 유효하지 않은지를 알려 주었고, C 구현을 통한 선택 비용 개선처럼 다음 작업의 방향이 되었다. 마지막으로 GPU가 없는 환경에서 재현성 검증 기능을 만들면서, 결과를 다른 사람이 다시 확인할 수 있게 만드는 것까지가 연구의 일부라는 점을 배웠다.

■ 참고문헌

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in Proc. Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.
- [2] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for Transformer-based generative models," in Proc. 16th USENIX Symp. on Operating Systems Design and Implementation (OSDI), 2022, pp. 521–538.
- [3] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in Proc. 29th ACM Symp. on Operating Systems Principles (SOSP), 2023, pp. 611–626.
- [4] T. Dao, D. Haziza, F. Massa, and G. Sizov, "Flash-Decoding for long-context inference," PyTorch Blog, Oct. 2023. <https://pytorch.org/blog/flash-decoding/>
- [5] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," in Proc. Int. Conf. on Learning Representations (ICLR), 2024.
- [6] A. Bakhoda, G. L. Yuan, W. W. L. Fung, H. Wong, and T. M. Aamodt, "Analyzing CUDA workloads using a detailed GPU simulator," in Proc. IEEE Int. Symp. on Performance Analysis of Systems and Software (ISPASS), 2009, pp. 163–174.
- [7] M. Khairy, Z. Shen, T. M. Aamodt, and T. G. Rogers, "Accel-Sim: An extensible simulation framework for validated GPU modeling," in Proc. ACM/IEEE 47th Annu. Int. Symp. on Computer Architecture (ISCA), 2020, pp. 473–486.
- [8] A. Yang et al. (Qwen Team), "Qwen3 technical report," arXiv:2505.09388, May 2025.
- [9] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, and R. Ramjee, "Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve," in Proc. 18th USENIX Symp. on Operating Systems Design and Implementation (OSDI), 2024.
- [10] 김수진, 유현창, "LLM 추론 성능 최적화를 위한 클러스터링 기반 KV 캐시 재사용 시스템," 한국정보처리학회 2025년도 추계학술발표대회 논문집, pp. 592–595, 2025.
- [11] 최우형, 정진우, 안정섭, "거대 언어 모델을 서빙하기 위한 GPU 메모리 계층화 사례 연구," 정보과학회지, 제 43 권 제 4 호, pp. 37–40, 2025.
- [12] 정선욱, 이성주, 강범우, 오테욱, 박영준, "다중 GPU 시스템에서의 복잡한 딥러닝 모델의 고성능 추론을 위한 스케줄링 전략 분석," 한국정보과학회 2022 한국컴퓨터종합학술대회 논문집, pp. 1592–1594, 2022.
- [13] 박시형, 이제민, 김병수, 전석훈, "대규모 언어 모델 가속을 위한 상용 가속기 기술 동향," 한국정보기술학회논문지, 제 23 권 제 6 호, pp. 75–89, 2025.
- [14] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in Proc. Advances in Neural Information Processing Systems (NeurIPS), vol. 35, 2022.
- [15] K. Hong, G. Dai, J. Xu, Q. Mao, X. Li, J. Liu, K. Chen, Y. Dong, and Y. Wang, "FlashDecoding++: Faster large language model inference with asynchronization, flat GEMM optimization, and heuristics," in Proc. 7th Conf. on Machine Learning and Systems (MLSys), 2024.
- [16] Z. Ye, L. Chen, R. Lai, W. Lin, Y. Zhang, S. Wang, T. Chen, B. Kasikci, V. Grover, A. Krishnamurthy, and L. Ceze, "FlashInfer: Efficient and customizable attention engine for LLM inference serving," in Proc. 8th Conf. on Machine Learning and Systems (MLSys), 2025.

- [17] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009.
- [18] 정종현, 오윤희, 구건재, "최신 GPU 아키텍처의 데이터 캐시 성능 분석," *한국정보과학회 2021 한국소프트웨어종합학술대회 논문집*, pp. 1052–1054, 2021.
- [19] 최진서, 강동현, "딥 러닝 연산을 위한 GPU/CPU 성능 분석," *한국정보과학회 2022 한국소프트웨어종합학술대회 논문집*, pp. 1145–1147, 2022.
- [20] 김광복, 김종면, 김철홍, "GPU 성능 향상을 위한 MSHR 정보 기반 워프스케줄링 기법," *한국차세대컴퓨팅학회 논문지*, 제 13 권 제 3 호, pp. 72–83, 2017.
- [21] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," in *Proc. 1st Conf. on Language Modeling (COLM)*, 2024.

■ 부록 A. 원천코드 (주요 부분 발췌)

전체 원천코드는 GitHub 저장소(<https://github.com/sunjae0224/kernelscope>, design-1-3 브랜치)에 있다. 아래는 본문에서 설명한 핵심 부분의 발췌이다.

A.1 FlashAttention 2.8.3의 분할 수 휴리스틱 재현 (kernelscope/model/geometry.py)

```
def num_splits_heuristic(batch_nheads_mblocks, num_sms, num_n_blocks, max_splits=128):
    """Exact port of flash-attn 2.8.3 num_splits_heuristic (flash_api.cpp)."""
    if batch_nheads_mblocks >= 0.8 * num_sms:
        return 1
    max_splits = min(max_splits, num_sms, num_n_blocks)
    eff = [0.0] * (max_splits + 1)
    best = 0.0
    for s in range(1, max_splits + 1):
        if s > 1 and math.ceil(num_n_blocks / s) == math.ceil(num_n_blocks / (s - 1)):
            continue
        n_waves = batch_nheads_mblocks * s / num_sms
        eff[s] = n_waves / math.ceil(n_waves)
        best = max(best, eff[s])
    for s in range(1, max_splits + 1):
        if (s == 1 or math.ceil(num_n_blocks / s) != math.ceil(num_n_blocks / (s - 1))) \
            and eff[s] >= 0.85 * best:
            return s
    return 1

def resolve_splits(v, w, n_sm):
    if v.num_splits > 0:
        return v.num_splits
    m_blocks = math.ceil((w.H_q // w.H_kv) / 64)  # GQA-packed query rows, 64 per m-block
    return num_splits_heuristic(w.B * w.H_kv * m_blocks, 2 * n_sm,
                               math.ceil(capacity(w, v.paged) / BLOCK_N), 128)
```

A.2 조회 테이블 정책 (kernelscope/serve/dispatch.py)

```
class TablePolicy:
    @staticmethod
    def _features(lens):
        return np.log2([len(lens), max(lens), float(np.mean(lens))])

    def choose(self, lens, n_heads, n_kv_heads) -> int:
        _validate(lens, n_heads, n_kv_heads)
        key = (n_heads, n_kv_heads, tuple(math.ceil(n / PAGE) for n in lens))
        self.last_cache_hit = key in self._cache
        if self.last_cache_hit:
            value, self.last = self._cache[key]
            return value
        eligible = self._groups.get((n_heads, n_kv_heads), [])
        if not eligible:
            raise ValueError(f"dispatch table has no measured shape with Hq={n_heads},
Hkv={n_kv_heads}")
        query = self._features(lens)
        distances = np.square(self._features_by_shape[(n_heads, n_kv_heads)] - query).sum(axis=1)
        index = int(np.argmin(distances))
        workload, value = eligible[index]
        self.last = {"workload_key": workload.key(), "distance": float(distances[index] ** 0.5)}
        self._cache[key] = (value, self.last)
```

```
return value
```

A.3 재현성 검증의 비교 절차 (kernelscope/verify.py, 이번 기간 신규)

```
def run_checks(checks, repo: Path, data: Path) -> pd.DataFrame:
    rows = []
    for c in checks:
        measured, note = math.nan, ""
        try:
            measured = float(c.compute(repo, data))
            status = "PASS" if abs(measured - c.expected) <= c.tol else "FAIL"
        except (FileNotFoundError, KeyError, StopIteration, IndexError) as e:
            status, note = "MISSING", f"{type(e).__name__}: {e}"
        if status == "PASS" and c.doc:
            path = repo / c.doc
            if not path.exists() or c.doc_text not in path.read_text():
                status, note = "DOC_DRIFT", f"{c.doc} no longer states {c.doc_text!r}"
        rows.append({"id": c.id, "expected": c.expected, "measured": measured, "status": status})
    return pd.DataFrame(rows)

# 예: attention 시간 변화만으로 예측한 단계 시간과 실측의 차이(%)
def _amdahl_error_pct(scenario, policy="table"):
    def f(repo, data):
        s = _summary(data, scenario)
        h, p = s.loc["heuristic"], s.loc[policy]
        predicted = h.step_ms_per_step - h.attn_ms_per_step + p.attn_ms_per_step
        return 100 * (predicted / p.step_ms_per_step - 1)
    return f
```

A.4 실행 방법

```
# GPU 없는 환경: 테스트, 재현성 검증, 보고서 그림, 대시보드
make test
make verify          # 37개 항목 재계산 및 대조
make figures         # docs/report/fig 에 보고서 그림 생성
make demo            # http://localhost:8501

# GPU 환경: 커널 측정과 실제 모델 실험
python -m kernelscope.cli bench --grid grids/ragged_s1.yaml \
    --plugins fa2,flashdecoding,fd_s2,fd_s4,fd_s8,fd_s16 --results
results/hw_4090/ragged_s1_dense \
    --cache-state cold,warm
python -m kernelscope.cli serve run --model Qwen/Qwen3-4B-Instruct-2507 \
    --scenario scenarios/graduation_ragged.yaml --policy heuristic --policy
table:demo_data/dispatch_paged_cold.csv
```